

Computational Simulation of Yin’s (1969) Face Inversion Effect

Using a Log-Polar Neural Network and a KDE Memory Model,
Averaged Across Four Trained Models

Siddharth Agrawal

July 18, 2026

Abstract

We test whether a biologically inspired neural memory model reproduces the classic result of Yin (1969): that turning a picture upside down hurts memory for faces far more than it hurts memory for other kinds of objects. We train a log-polar (foveated) convolutional network from scratch on faces, objects, and houses, read out its internal binary code, and feed that code into a Kernel Density Estimation (KDE) familiarity model based on the Barrington NIMBLE framework. We then run Yin’s exact study-then-test recognition design. Averaging over four independently trained models, the network shows a large inversion cost for faces and only a small one for objects, matching both the pattern and roughly the size of Yin’s human data.

1 Introduction and Background

Yin (1969) is one of the foundational studies in face recognition. It showed that human memory for faces is hurt much more by turning the picture upside down than memory for other familiar, complex objects that are also normally seen in only one orientation (“mono-oriented”), such as houses, airplanes, and stick figures. This selectivity became a key piece of evidence that face recognition relies on a special, *holistic* (whole-at-once) processing mechanism, rather than the generic strategies we use for ordinary objects.

Yin’s task was a forced-choice recognition memory test. Each participant first saw an **inspection (study) series** of 40 pictures, shown one at a time for 3 seconds each. They then saw a **test series** of 24 pairs of pictures. Every pair contained exactly one *old* picture (an exact copy of one seen during study) and one *new* picture (never shown before), and the participant simply pointed to the one they had seen before. Only 24 of the 40 studied pictures came back in the test, so 16 were never re-tested — a realistic memory situation rather than a memorize-everything drill.

Yin ran the study in two orientation setups. **Experiment I** kept the orientation the same across study and test: either both upright (Upright–Upright) or both inverted (Inverted–Inverted). **Experiment II** flipped the orientation between the two phases: study upright and test inverted (Up–Down), or study inverted and test upright (Down–Up). The main finding was that *every* category got harder when inverted, but faces got disproportionately harder — pointing to one general “familiar orientation” factor that affects all objects, plus a second, special factor that applies only to faces.

This report reproduces Yin’s design computationally. We use a neural network as a stand-in for the visual system that turns a picture into an internal representation, and a simple memory model that decides whether a test picture feels “familiar.” Two design choices are worth noting up front: we average results over four separately trained networks instead of relying on one, and we report the object category alongside faces so the dissociation can be seen directly.

2 Converting Yin’s Error Scores to Accuracy

Yin reported performance as the **mean number of errors** out of 24 test pairs. Our model outputs a percentage accuracy, so to compare the two we convert Yin’s error scores to accuracy with

$$\text{Accuracy}(\%) = \frac{24 - M_{\text{errors}}}{24} \times 100.$$

Accuracy and error count carry the same information here (one is just 24 minus the other, rescaled), so the inversion effect can equally be read as a difference in error counts; we use accuracy below for a direct side-by-side with the model. Table 1 gives the converted human values for the two categories we compare against: faces, and airplanes (which we treat as the human “object” comparison, since our network’s object set is closest in spirit to Yin’s simple, unfamiliar object caricatures).

Table 1: Yin (1969) mean errors (out of 24) with subject-level standard deviations, and the derived accuracy, for the faces and airplane (“object”) categories across all four orientation conditions. Error *SD* values are Yin’s originals (Exp. I Table 1 and Exp. II Table 2, across $n = 26$ and $n = 21$ subjects). The accuracy *SD* is the error *SD* rescaled by $100/24$, since accuracy is a linear transform of error count and so carries the same dispersion.

Category	Study → Test	Mean Errors (<i>SD</i>)	Derived Accuracy (<i>SD</i>)
Faces	Upright → Upright	0.89 (1.09)	96.29% (4.5)
Faces	Inverted → Inverted	4.35 (1.41)	81.88% (5.9)
Faces	Upright → Inverted	3.81 (1.71)	84.13% (7.1)
Faces	Inverted → Upright	5.14 (1.39)	78.58% (5.8)
Airplanes	Upright → Upright	3.65 (1.69)	84.79% (7.0)
Airplanes	Inverted → Inverted	3.85 (2.03)	83.96% (8.5)
Airplanes	Upright → Inverted	3.19 (1.94)	86.71% (8.1)
Airplanes	Inverted → Upright	4.14 (1.58)	82.75% (6.6)

Even before any modeling, the human numbers already show the effect: faces drop about **15–18 points** when inversion is involved, while airplanes barely move (within about 2 points, and in one condition they even improve).

3 Computational Simulation: Experimental Design

3.1 Model Architecture and Feature Extraction

The neural model is a convolutional feature extractor in the LPNet style: each fixation is first foveated (sharp in the center, blurred toward the edges) and then passed through a *log-polar* transform, which mimics how the retina samples the visual field. Fixation points themselves are chosen bottom-up from a bank of Gabor filters (a model of V1 simple cells), so the same, non-face-specific mechanism is used for every category.

The network is trained from scratch (no borrowed pretraining) to classify a combined set of **128 face identities, 64 object categories, and houses** (pooled into one generic “house” class), all under a single shared classification head. Each training sample is a 180×180 patch cropped around one of the Gabor-selected fixation points; before foveation and the log-polar transform, the patch is randomly rotated by up to $\pm 15^\circ$ (a fresh angle every epoch, modelling small natural variations

in head and gaze orientation) and horizontally flipped with probability 0.5. The flip is horizontal only: a vertical flip is never applied during training, since that would leak inverted views into the network and contaminate the very inversion manipulation the simulation is designed to test. After a bottleneck layer, it produces a stochastic **binary hidden representation** $h \in \{0, 1\}^{256}$ through a temperature-scaled sigmoid followed by Bernoulli sampling — directly analogous to a binary code over a population of simulated neurons. The classifier head is only used to *train* the network; the memory model reads the shared 256-dimensional code h .

To keep the results from depending on any single trained network, we **average over four independently trained models**, spanning two backbone sizes (ResNet-18 and ResNet-34) and two training fixation counts (16 and 32 fixations per image). Every number reported in the results tables is a mean across these four models; we never report a single model on its own.

3.2 Simulating the Study Phase (Inspection Series)

Mirroring Yin’s 40-picture inspection series, 40 items are loaded for the study phase. For each item, 10 fixation patches are extracted — the spots where a viewer’s gaze might land during the brief 3-second exposure — and passed through the network to get its binary code. Each item’s code is stored in a **memory bank**, the computational analogue of encoding a picture into episodic memory.

To add biological realism, **binomial noise** is applied to each stored code by randomly flipping bits:

$$h_{\text{noisy}} = h \oplus m, \quad m_i \sim \text{Bernoulli}(p_{\text{noise}}).$$

This models the fact that a memory is not a perfect digital recording but a rough, partly scrambled impression.

3.3 Simulating the Test Phase (24 Forced-Choice Pairs)

Matching Yin’s 24-pair test series, 24 *old* items are drawn from the 40 studied ones and paired with 24 *new* items never seen during study. Each test item is viewed through 32 fixation patches (a richer look, reflecting the slower, self-paced test phase). Noise is again applied to these retrieval codes, modelling the fact that pulling up a memory is itself imperfect.

A **familiarity score** is then computed for each test item using a Kernel Density Estimation (KDE) rule from the Barrington NIMBLE framework:

$$p(f \mid c) = \frac{1}{|M_c|} \sum_{m \in M_c} \exp\left(-\frac{\|f - m\|^2}{2\sigma^2}\right),$$

where f is the code from a single test fixation, M_c is the set of stored codes for a studied item c , and σ is the kernel bandwidth (how forgiving the similarity is). The familiarity of a whole test item is the best (highest log-likelihood) match against any stored item. The decision rule follows Yin exactly: **if the old item scores higher than the new item, the trial is correct.**

3.4 Noise Calibration

The noise level p_{noise} was calibrated separately for each model and each category on its own Upright–Upright condition, choosing the smallest noise that brings model accuracy down to the matching human baseline. For faces the target is Yin’s human faces baseline ($\approx 96\%$), and across the four models this landed between $p = 0.00$ and $p = 0.15$ (mean ≈ 0.06). For objects the target is the

human object (airplane) baseline of 84.79%; here the models were already at or below this level with no added noise, so objects were run at $p = 0.00$. In every case the calibrated noise was then held fixed across all four orientation conditions for that model.

3.5 Why the Noise

Noise enters at two points, and both are intentional. During the **study phase**, the model glances at each item through 10 fixations and stores a corrupted version of what it sees — the idea that a few seconds of looking leaves a rough impression, not a photograph. During the **test phase**, each item is viewed through 32 fixations and noise is applied again before any comparison — the separate idea that *retrieving* a memory is itself imperfect, so even a well-stored trace does not come back fully intact. Using one noise level at both stages keeps the model simple, and tying its value to the human Upright–Upright baseline means the noise is grounded in the data rather than chosen arbitrarily.

4 Results

4.1 Inversion Effect During Training

Before the memory simulation, the inversion effect is already visible in the trained network’s own classification accuracy: we simply measure how well each model classifies upright pictures versus the same pictures inverted (a 180° rotation applied before the log-polar transform). Table 2 shows the mean across the four models.

Table 2: Classification accuracy on upright vs. inverted pictures, averaged across the four trained models. “Drop” is the accuracy lost to inversion.

Category	Upright	Inverted	Drop
Faces	93.9%	61.2%	−32.7
Houses	95.8%	74.5%	−21.3
Objects	77.2%	72.0%	−5.2

This is exactly the ordering Yin found. Faces lose about **33 points** to inversion, houses about **21 points**, and objects only about **5 points**. The face-specific cost is roughly six times the object cost, with houses falling in between — a clean face > houses > objects hierarchy that echoes Yin’s “general familiarity factor plus a special face factor.”

4.2 Results of the Yin Simulation

We now report the full Yin 2AFC memory simulation, again as the mean across the four models. We show **faces** and **objects**. (Houses are left out here: because all houses were trained as a single generic class, the network’s code cannot tell one house from another, so the old/new memory test for houses is at chance and not meaningful. Houses remain useful for the training-accuracy comparison above, where per-house identity is not required.)

Table 3 gives faces and Table 4 gives objects. Alongside the model and human accuracies, we list the **inversion cost**: how far each condition falls below its own Upright–Upright baseline. This “difference between rows” is the number that makes the dissociation visible.

Two things should be read carefully alongside these tables. First, **the Upright–Upright row is fit, not predicted.** Because p_{noise} is calibrated on each model’s own Upright–Upright condition to match the human baseline (Section 4.4), the agreement in the UU row is true by construction and is *not* independent evidence. The genuine model predictions are the three inversion costs — the Model Δ column — which are what should be compared against the human Δ . Second, **the human “object” column is Yin’s airplanes.** Our network’s object category is a generic 64-class object set, whereas Yin’s closest mono-oriented object comparison is airplanes; we therefore compare model objects against human airplanes throughout. The two are not the same stimuli, so the object comparison is a match of the overall *pattern* (small, roughly flat inversion cost) rather than a stimulus-for-stimulus fit. Finally, the model SD is computed across only four trained models and each condition is scored on 24 pairs, so the standard deviations are large relative to the object Δ s: the face inversion cost clears its own SD , while the 2–5 point object costs sit within it.

Table 3: Faces: model accuracy (mean of four models, with the standard deviation across those four models in parentheses) vs. Yin’s human accuracy (subject-level SD in parentheses), with the inversion cost relative to Upright–Upright.

Study	Test	Model Acc. (SD)	Human Acc. (SD)	Model Δ	Human Δ
Upright	Upright	95.8% (0.0)	96.29% (4.5)	—	—
Inverted	Inverted	80.2% (5.2)	81.88% (5.9)	−15.6	−14.4
Upright	Inverted	78.1% (6.3)	84.13% (7.1)	−17.7	−12.2
Inverted	Upright	76.0% (6.3)	78.58% (5.8)	−19.8	−17.7

Table 4: Objects: model accuracy (mean of four models, with the standard deviation across those four models in parentheses) vs. Yin’s human *airplane* accuracy (subject-level SD in parentheses; airplanes are Yin’s object category we compare against, see text), with the inversion cost relative to Upright–Upright.

Study	Test	Model Acc. (SD)	Human Acc. (SD)	Model Δ	Human Δ
Upright	Upright	80.2% (6.3)	84.79% (7.0)	—	—
Inverted	Inverted	75.0% (3.4)	83.96% (8.5)	−5.2	−0.8
Upright	Inverted	77.1% (5.4)	86.71% (8.1)	−3.1	+1.9
Inverted	Upright	78.1% (4.0)	82.75% (6.6)	−2.1	−2.0

Qualitative fit

The model reproduces the main qualitative pattern. For **faces**, Upright–Upright is the best condition, Upright–Inverted is worse than that and then Inverted–Upright is the worst.

For **objects**, all four conditions sit close together (within about 5 points for the model, within about 4 for the humans): inversion barely matters.

Comparing faces to objects, the accuracy drop when the images are inverted is much worse in faces (16–20pt drop) vs in objects (2–5).

Quantitative fit

The match is not just qualitative. For **faces**, the inversion costs are close in size: the model loses 15.6, 17.7, and 19.8 points across the three inverted conditions, against the human 14.4, 12.2, and 17.7. Upright–Inverted is a weak spot, where the model over-estimates the cost (it makes studying upright then testing inverted a little harder than humans do), and it slightly swaps the order of the two middle conditions. But the overall size of the face inversion effect is reproduced, not just its direction.

For **objects**, the model’s inversion cost stays small (2–5 points) just as the human effect does (0–2 points). The model runs a few points below the human accuracy overall, mainly because objects are a genuinely harder classification problem for the network, which is why they already sat at their human baseline with no added noise. Importantly, this works *against* finding a dissociation — objects are the harder task here, yet still show the smaller inversion cost — so the contrast between faces and objects is if anything understated.

Summary of the dissociation

Putting the two tables together: inversion “hurts” faces by roughly 16–20 points and “hurts” objects by only 2–5 points, both in line with the human data. The special cost that Yin attributed to faces shows up in the model even though nothing about faces was built into the fixation mechanism or the memory rule — the same pipeline was used for every category.

5 Conclusion

A from-scratch log-polar network, read out through a simple KDE familiarity memory, reproduces Yin’s (1969) central result: memory for inverted faces is disproportionately impaired compared with memory for inverted objects. The effect appears twice over — in the network’s raw classification accuracy (faces lose ~ 33 points to inversion, objects only ~ 5) and in the full study-then-test memory simulation (a 16–20 point face cost against a 2–5 point object cost), with both the pattern and the rough magnitude matching the human data. Because the model treats faces, objects, and houses with one shared mechanism, the emergence of a face-selective inversion cost suggests that at least part of Yin’s effect can arise from generic foveated, orientation-sensitive visual processing rather than requiring a hand-built, face-only module.

References

- Barrington, L., Marks, T. K., & Cottrell, G. W. (2008). NIMBLE: A kernel density model of saccade-based visual memory. *Journal of Vision*, 8(14), 1–14.
- Yin, R. K. (1969). Looking at upside-down faces. *Journal of Experimental Psychology*, 81(1), 141–145.